

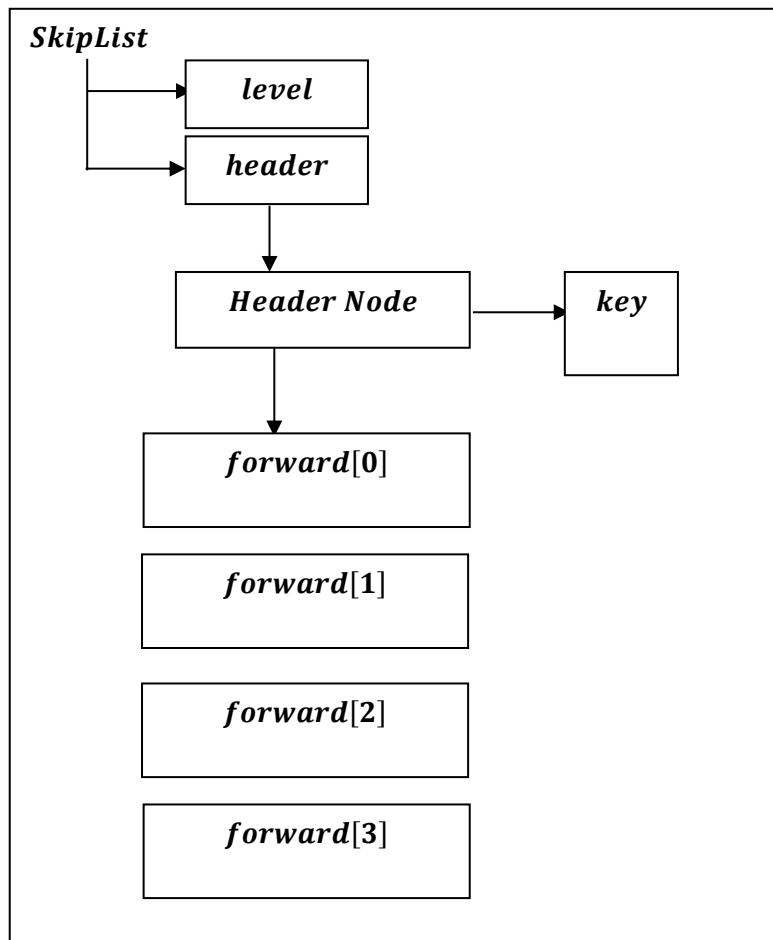
Skip List – Insert Element

–Insert Module Summary

```
void insertElement(struct SkipList *lst, int key)
```

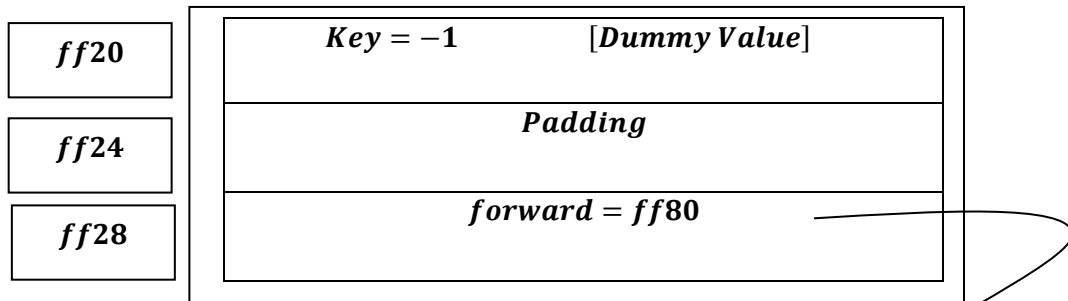
It receives two things:

- *lst* → *pointer to the Skip List.*
- *key* → *value that we want to insert.*

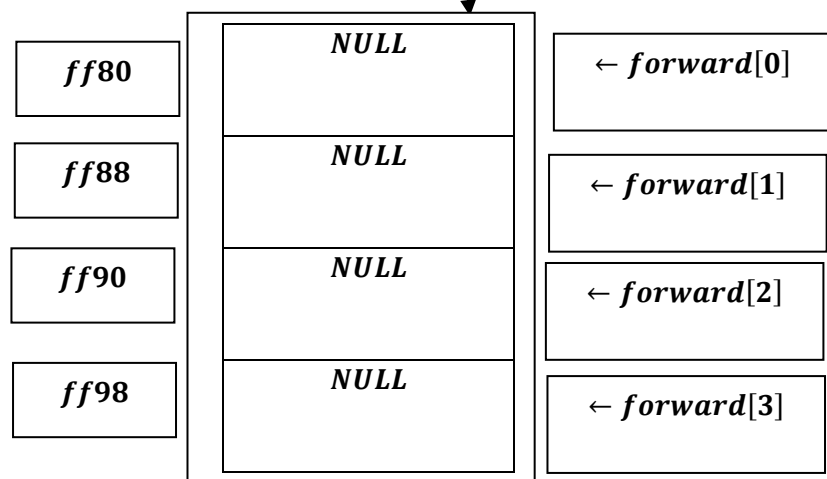


More Precisely:

Header Node or Dummy Node [address: ff20]

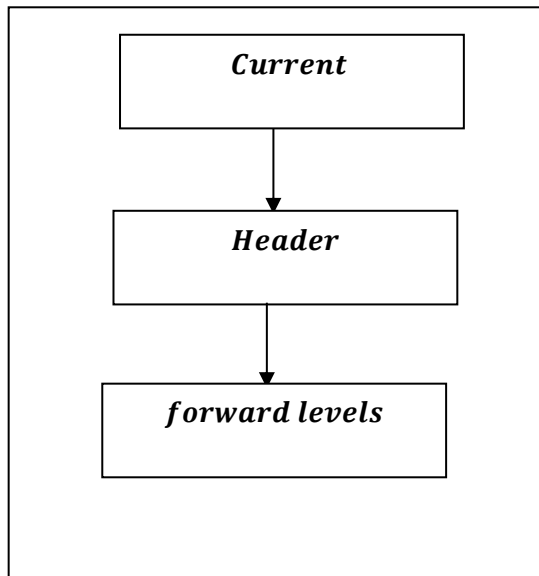


***Forward pointer array starts
at address ff80***



II. Initialize Current

```
struct Node *current = lst->header;
```



Why we need current ?

Because the algorithm must walk through the Skip List to find where the new key belongs.

- **To Initialize the Search:**

The current pointer acts as the active navigator or cursor for the list, explicitly starting its journey at the header node of the skip list (`struct Node *current = lst->header;`).

- **To Traverse and Find the Insertion Point:**

It is used within a while loop to physically move horizontally across the nodes at each level. The program compares the next node's key against the target key being inserted (`current->forward[i] -> key < key`). If the next key is smaller, the current pointer steps forward to that node (`current = current->forward[i]`).

- **To Record the Pointer Path:**
Once current cannot move any further right on a given level, its exact address is saved into the update array (`update[i] = current`). This creates a map of the path taken, ensuring the program knows precisely which previous nodes must be rewired to point to the newly inserted node later on.
- **To Prevent Duplicates:**
After dropping down through all the levels, the program moves current one final step forward on level 0 (`current = current → forward[0]`). This lands current exactly where the new key is supposed to go. The pointer is then used to validate whether that key already exists in the list (`if (current == NULL || current → key != key)`) to enforce unique keys and prevent duplicate insertions.

III. Initialization and the update Array

Before inserting a new element, the program must track where the new node will link into the existing list.

- The function initializes an `update` array to keep track of the path taken and the nodes that will need their pointers updated.
- The size of this array is calculated as `MAX_LEVEL + 1`.
- Because `MAX_LEVEL` is 3, the array holds exactly 4 elements.
- When initially declared, the array holds random garbage data (wild pointers) on the stack.
- To prevent undefined behavior, the `memset` function is used to rapidly fill this entire memory block with 0 or NULL values.

Why we need Update ?

→ To Track Pointer Updates:

Its primary purpose is to keep track of the specific path taken during the search, logging the exact nodes that will need their pointers updated.

→ To Mark the "Drop" Points:

As the **current** pointer searches for the correct insertion spot, it moves rightwards and then drops down level by level. At each level **i**, right before dropping down, the program saves the current node's address into the array (**update[i] = current;**). This essentially records the "predecessor" node for the new element at every single level.

→ To Handle New Maximum Levels:

If the new node generates a random level (**rlevel**) that is taller than the current list, the **update** array is used to link those brand – new upper levels directly to the list's header:
(**update[i] = lst->header;**).

→ To Execute the Final Rewiring:

Once the new node (**n**) is created in memory, the program relies entirely on the **update** array to connect it. It uses the saved predecessor nodes to point the new node forward

(**n → forward[i] = update[i] → forward[i]**), and then rewires those predecessor nodes to point directly to the new node

(**update[i] → forward[i] = n**).

IV. The core search and mapping engine for the Skip List.

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

- *for (int i = lst → level; i ≥ 0; i --): This initiates a top – down search.*

Instead of searching one by one from the bottom,

it starts at the highest current level of the list (lst → level)

and drops down step – by – step to level 0 (i – –). This is what gives a

skip list its speed, allowing it to use the upper levels as express lanes

to skip over large chunks of data.

- *while (current → forward[i] != NULL
&& current → forward[i] → key < key):*

At each level, the program looks ahead to the next node.

It checks two critical things before moving:

1. *!= NULL: Does a next node actually exist, or have we reached the end of the list?*
2. *→ key < key: Is the next node's key smaller than the one we are trying to insert?*

- *`current = current → forward[i];` If the next node exists and has a smaller key, the current pointer moves horizontally to the right. This while loop continues moving the pointer rightward until it hits a node that is larger, or hits the end of the list.*
- *`update[i] = current;` This is the most crucial part for the actual insertion. When the `current` pointer can no longer move forward on level `i`, it stops.*

The program then records this exact node into the `update` array (`update[i] = current`) before the `for` loop forces the pointer to drop down to the next level.

V. Duplicate Key Validation

Skip lists often act as sets or dictionaries, meaning they require unique keys.

- *After navigating down to level 0, the program steps forward one final time using `current = current → forward[0]` to inspect the exact spot where the new key belongs.*
- *The program validates uniqueness using the condition*
`if (current == NULL || current->key != key).`
- *If `current` matches the key we are trying to insert, the condition fails, the insertion block is skipped, and the program prints that the key already exists.*
- *While this implementation strictly blocks duplicates, standard computer science skip lists could theoretically allow them by removing the `!= key` check.*

VI. Random Level Generation and Header Adjustment

If the key is unique, the program prepares to build the new node.

- *A random level (**rlevel**) is generated for the new node via the **randomLevel()** function.*
- *The program must account for cases where the generated **rlevel** is higher than any node currently in the list (**rlevel** > **lst** → **level**).*
- *If **rlevel** is greater than the current **level** a **for loop** updates the new highest slots in the **update** array to point directly to the list's **header**.*
- *The list's overall maximum level tracker (**lst** → **level**) is then permanently updated to match this new **rlevel**.*

*If the randomly generated level (**rlevel**) for the new node is strictly greater than the list's current level (**lst** → **level**), the program executes a specific if block to safely increase the **height of the skip list**.*

Here is exactly what happens in that scenario:

- ***Anchoring to the Header**: A **for** loop iterates from the first new level (**lst** → **level** + 1) up to the new **rlevel**. Because these newly generated upper levels are completely empty and have no existing predecessor nodes, the program must set the **update** array for these levels to point directly to the list's **header** node (**update[i]** = **lst** → **header**).*

```
for (int i = lst->level + 1; i <= rlevel; i++) {
    update[i] = lst->header;
}
```

- **Updating the Global Tracker:** After initializing the **update** array for the new levels, the program updates the skip list's internal property that tracks its height. It permanently sets **lst → level = rlevel**, ensuring that all future searches and insertions know the list has grown taller.

```
lst->level = rlevel;
```

Exceeding the Current Max Level (lst → level): If the new node's randomly generated level (**rlevel**) is higher than the tallest node currently existing in the list, the program gracefully adapts. Instead of crashing, it safely anchors those brand – new upper levels to the list's **header** and securely updates the list's internal height tracker (**lst → level = rlevel**).

```
for (int i = lst->level + 1; i <= rlevel; i++) {
    update[i] = lst->header;
}
lst->level = rlevel;
```

The Absolute Hard Limit (MAX_LEVEL): The program will never actually exceed the absolute hard cap defined by **MAX_LEVEL** (which evaluates to 3 in this program). The **randomLevel()** function includes a strict built – in safety check:

while ((rand() % 2) == 0 && lvl < MAX_LEVEL). This condition guarantees that the loop generating the random level immediately stops incrementing if it hits the **MAX_LEVEL** cap.

```

int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}

```

VII. Node Creation and Memory Allocation

```

    struct Node *n = createNode(key, rlevel);
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

```

Once the level is finalized, memory must be explicitly allocated for the new data.

- The **createNode(key, rlevel)** function uses **malloc** to reserve memory space.
- It allocates 8 bytes to hold the primary **Node** structure
(which stores the integer key itself).
- It dynamically allocates memory for the **forward** array based on the level, taking **(level + 1) * 8 bytes**.

- A **for loop** inside **createNode** ensures that all freshly allocated forward pointers are initially set to **NULL** before returning the new node **n**.

VII. Pointer Rewiring (Insertion)

```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
cout << "Successfully inserted key " << key << endl;
```

The final step stitches the newly created node into the fabric of the existing list.

- A loop iterates from level 0 up to the node's rlevel (**i <= rlevel**).
- First, the new node's forward pointers take on the destinations previously held by the **update** array:
n->forward[i] = update[i]->forward[i].
- Next, the nodes stored in the **update** array drop their old connections and point directly to the new node:
update[i]->forward[i] = n.
- Once rewiring is complete, the program outputs a **success message and the insertion concludes**.

The Skip List's ``Skip`` mechanism in insertion

The skip in a Skip List happens during the very first phase of the insertion process: the search phase.

Before a new node can be inserted, the program must figure out exactly where it belongs. Instead of checking every single node one by one (like a standard linked list), it uses the upper levels to "skip" over large sections of the data.

Here is exactly where and how this skipping occurs in the provided program:

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

- **Starting in the Express Lane:** The skip is initiated by the loop `for (int i = lst->level; i ≥ 0; i--)`. The program explicitly states it will "Start from the highest current level and move down". By starting at `lst->level` rather than `level 0`, the pointer begins its search on the tallest available structural layer.
- **Skipping Nodes Horizontally:** Inside that for loop is the while loop: `while (current->forward[i] != NULL && current->forward[i]->key < key)`. When `i` is a high number (like 2 or 3), the command `current = current->forward[i]` moves the pointer to the next node on that specific upper level.

- **The Actual Skip:** Because nodes at higher levels are spread further apart, a single horizontal jump at **level 2 or 3** might physically bypass (or skip) multiple smaller nodes that only exist down on **level 0**.
- **Dropping Down to Fine – Tune:** When the pointer can no longer skip forward on the current high level (because the next node is either **NULL** or its **key** is larger than the **target key**), the **while** loop exits. The **for** loop then triggers **i – –**, causing the pointer to drop down to the next level to continue a more fine – grained search.

By repeatedly skipping horizontally on high levels and dropping down when necessary, the program rapidly homes in on the correct insertion spot without having to evaluate every single key in the list.

Note: –

```
int randomLevel() {
    int lvl = 0;
    // 50% probability to increase the level
    while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
        lvl++;
    }
    return lvl;
}
```

Here, **MAX_LEVEL = 3**, according to the program:

say 1st, rand() = 42

1st loop run:

while $\left((rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \right)$:
AND lvl = 0 < MAX_LEVEL = 3 [TRUE]

lvl = 0 + 1 = 1

2nd, rand() = 998

2nd loop run:

while $\left((rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \right)$:
AND lvl = 1 < MAX_LEVEL = 3 [TRUE]

lvl = 1 + 1 = 2

3rd, rand() = 8470

3rd loop run:

while $\left((rand() = 8470 \% 2) = 0 == 0 \left[\frac{8470}{2} = 0[Remainder] \right] [TRUE] \right)$:
AND lvl = 2 < MAX_LEVEL = 3 [TRUE]

lvl = 2 + 1 = 3

4th, rand() = 1070

3rd loop run:

while $\left(\begin{array}{l} (rand() = 1070 \% 2) = 0 == 0 \left[\frac{1070}{2} = 0[Remainder] \right] [TRUE] \\ \text{AND } lvl = 3 < MAX_LEVEL = 3 [False] \end{array} \right)$

[While Loop Exits]

return, lvl = 3

∴ Maximum lvl or level or rlevel, can be `3` as in program, it is:

```
#define MAX_LEVEL 3
```
